AI–ML–DL Concept Tutor

An intelligent multi-level educational tutor and interactive research lab spanning Foundations of Generative AI, GANs, Large Language Models, Multimodal Systems, and Responsible AI.

Name	Aarush Parate
Class	LY AIA (B.Tech Computer Science)
Enrolment No.	ADT23SOCB0001
Roll No.	01
Subject	Generative AI

PROJECT REPORT — WITH DEPLOYMENT EVIDENCE

1. Problem and Objective

Project title: AI–ML–DL Concept Tutor (P1). The project objective is to construct an intelligent, multi-level educational tutor and interactive research laboratory called **NeuroTutor AI** that explains foundational Artificial Intelligence, Machine Learning, and Deep Learning concepts across three distinct cognitive knowledge levels (School Level, Undergraduate Level, and Expert Level), explores prompt engineering paradigms, extracts real attention weight matrices, ingests academic notes for grounded retrieval, interprets architectural schematics, generates educational illustrations, and evaluates answer reliability with verifiable grounding guardrails.

The P1 assignment specifies five mandatory syllabus units: (1) multi-level concept explanations with real-world analogies, algorithmic breakdowns, and mathematical formalisms; (2) prompt engineering comparison lab (Zero-shot, Few-shot, Chain-of-Thought) alongside softmax temperature dynamics; (3) conversational session memory and Hugging Face DistilBERT 72-head attention weight extraction; (4) academic notes ingestion (PDF, DOCX, TXT) with Sentence-BERT embeddings, FAISS vector search, and multimodal diagram parsing; and (5) measurable hallucination detection, safety guardrails, and uncertainty refusal. This report records the observed implementation evidence.

Project objectives

1. Convert complex AI/ML/DL topics into tailored conceptual explanations across School, Undergraduate, and Expert levels. 2. Provide an empirical prompt laboratory comparing Zero-Shot, Few-Shot, and Chain-of-Thought reasoning with dynamic softmax logit scaling. 3. Extract and visualize real DistilBERT multi-head self-attention heatmaps across 6 layers and 12 heads. 4. Ingest and semantically index academic lecture notes using Sentence-BERT and FAISS for grounded retrieval-augmented generation. 5. Support multimodal educational learning via architectural diagram parsing, diffusion concept illustration synthesis, and gTTS spoken audio recaps. 6. Implement verifiable claim grounding evaluation, adversarial prompt injection defense, and automated uncertainty refusal.

Scope and interpretation

The application is an educational tutor and interactive exploratory lab, not an autonomous high-stakes decision engine, medical diagnostic system, or commercial exam proctor. The 2D visual projections and attention heatmaps reflect real mathematical vector representations during forward inference. Grounding scores and uncertainty indicators represent probabilistic heuristic verification against retrieved note chunks.

Overall workflow

User queries and notes are ingested and processed by dedicated model pipelines. Conceptual prompts are compiled for the selected cognitive tier; prompt engineering variations are compared side-by-side; real DistilBERT attention weight tensors are extracted and mapped to interactive heatmaps; academic notes are parsed, chunked, and embedded into FAISS; multimodal schematics are analyzed; and generative outputs are cross-verified against reference evidence. Output cards, interactive charts, audio players, and metric tables are rendered dynamically in Streamlit.

Core application flow: input -> level & prompt compilation -> embedding / transformer inference -> FAISS retrieval / attention extraction / autograd -> multimodal synthesis / verification -> result interpretation.

2. Tools, Models and Dataset

Component	Technology / model	Role in the project
Programming & interface	Python 3.13; Streamlit 1.51	Application logic and modern SaaS user interface.
LLM inference & fallback	Google Gemini 2.5 Flash / Groq Llama 3.3 / Local Engine	Generates multi-tier explanations, prompt responses, and grounded synthesis.
Sentence embeddings	Sentence-Transformers: all-MiniLM-L6-v2	Encodes academic notes into 384-dimensional dense semantic vectors.
Vector search & similarity	FAISS CPU (IndexFlatIP); Cosine similarity	Indexes dense chunks and executes high-speed inner product similarity search.
Attention extraction	Hugging Face Transformers: DistilBERT	Extracts real attention weight matrices across 6 layers and 12 heads (72 subspaces).
Generative model	Project-defined PyTorch Minimax GAN	Learns a continuous bimodal Gaussian distribution through adversarial competition.
Neural network autograd	Project-defined 3-Layer PyTorch MLP	Demonstrates transparent forward activations, cross-entropy loss, and gradient backprop.
Document processing	pypdf; python-docx	Extracts raw text from PDF, DOCX, and TXT files and performs sliding-window chunking.
Multimodal vision	Gemini Vision / BLIP Heuristics	Parses architectural schematics and provides structural component breakdowns.
Concept illustration	Hosted Diffusion Inference Pipeline	Generates educational concept diagrams from expanded prompt descriptions.
Audio speech synthesis	gTTS (Google Text-to-Speech)	Synthesizes spoken audio explanations with regional accent controls.
Charts & data handling	Plotly; Pandas; NumPy; scikit-learn	Interactive heatmaps, loss curves, confusion matrices, and tabular metrics.

Recorded development configuration

The tested development environment was Python 3.13. Pinned package versions in the repository include Streamlit 1.51.0, PyTorch 2.9.0 (CPU), Transformers 4.57.1, Sentence-Transformers 5.1.0, FAISS-CPU 1.12.0, and Google GenAI SDK 1.0.0. Local development used CPU-only execution for DistilBERT attention extraction, FAISS vector indexing, and PyTorch autograd computational graphs.

GAN training configuration

Parameter	Recorded value
Input Distribution	Continuous 1D scalar sampled from bimodal Gaussian: 0.5 N(-2.5, 0.5^2) + 0.5 N(2.5, 0.5^2)
Latent Dimension (z)	4 dimensions: standard normal vector z ~ N(0, I_4)
Generator Architecture	Linear(4 -> 32) + ReLU -> Linear(32 -> 32) + ReLU -> Linear(32 -> 1) (Continuous)
Discriminator Architecture	Linear(1 -> 32) + LeakyReLU(0.2) -> Linear(32 -> 32) + LeakyReLU(0.2) -> Linear(32 -> 1) + Sigmoid
Optimiser & Learning Rate	Adam optimizer, lr = 0.002, betas = (0.5, 0.999)
Batch size / epochs	64 samples per batch / 25 interactive training epochs per step
Objective Loss Function	Adversarial Binary Cross-Entropy minimax loss V(D, G)

Prompting and model-learning comparison

Zero-shot prompting: Ask the generative LLM to complete a task or explain an AI concept using only direct task instructions, without providing exemplary demonstrations. Zero-shot prompting relies entirely on the pre-trained internal parametric knowledge of the transformer model. It incurs minimal prompt token consumption and lower latency, but can occasionally lack structured formatting or omit critical mathematical constraints on complex queries.

Few-shot prompting: Provide one or more structured input-output demonstration pairs within the context window prior to posing the actual target question. In-context exemplars guide the model regarding the desired technical vocabulary, mathematical notations, and structural section headers. In the Prompt Engineering Lab, Few-Shot prompting consistently produces more uniform explanations with predictable subsections, albeit with a ~35% higher prompt token footprint.

Chain-of-Thought (CoT) prompting: Directs the model to generate explicit step-by-step intermediate reasoning paths before arriving at the terminal answer (e.g. *'Let's think step by step: Step 1... Step 2... Step 3...'*). This technique forces the model to allocate dedicated computation tokens to intermediate deductive deductions, significantly reducing logical errors on complex deep learning algorithms such as backpropagation and self-attention derivations.

Prompt-based inference vs. fine-tuning

In **prompt-based learning**, underlying foundation model parameters remain completely frozen ($\Delta W = 0$). Behavioral adaptation, task conditioning, and tone moderation occur purely in-context through conditioning on prompt tokens at inference time. This allows instant multi-task switching without catastrophic forgetting or expensive re-training cycles.

Conversely, **fine-tuning** updates the model's weight matrices through supervised fine-tuning (SFT), LoRA adapter rank updates, or RLHF using backpropagation on domain datasets. This project utilizes prompt-based inference with dynamic session memory for its core tutor and prompt lab, while its local PyTorch modules (GAN and MLP) undergo genuine gradient-based backpropagation updates.

Model family note: GPT vs BERT architectural comparison

Transformers bifurcate into two fundamental architectural paradigms based on attention masking and training objectives:

1. **BERT Family (Bidirectional Encoder):** Utilizes unconstrained bidirectional self-attention where every token attends to all past and future positions simultaneously ($M_{ij} = 1$ for all i, j). Pre-trained via Masked Language Modeling (MLM). Sentence-BERT (all-MiniLM-L6-v2) and DistilBERT belong to this family, excelling at dense semantic representation extraction, semantic search, and clustering.

2. **GPT Family (Autoregressive Decoder):** Utilizes a causal lower-triangular attention mask ($M_{ij} = -\infty$ for $j > i$) preventing tokens from attending to future positions. Pre-trained on next-token prediction ($\max \Sigma_t \log P(x_t \mid x_{<t})$). Generative foundation models (Gemini, Llama, GPT-4) belong to this family, optimized for open-ended text synthesis and conversational reasoning.

3. Approach with Block Diagram

The system is organized into modular pipelines coordinating multi-level conceptual tutoring, prompt engineering comparison, real transformer attention extraction, academic notes RAG, multimodal vision, PyTorch autograd, and grounding verification.

SYSTEM ARCHITECTURE AND END-TO-END PROJECT LIFECYCLE

USER INPUT: Questions, Lecture Notes (PDF/DOCX/TXT), Schematics, Latent Parameters
↓
TIER 1 (Foundations): Prompt Lab (Zero/Few/CoT) • Softmax Temperature Simulator • Latent Manifold Projection
TIER 2 (Language & GANs): Multi-Tier Tutor • Sliding Memory • DistilBERT 72-Head Attention • PyTorch Minimax GAN
TIER 3 (RAG & Multimodal): Sentence-BERT (384-d) • FAISS Inner-Product Search • Diagram Explainer • Diffusion Studio
TIER 4 (Verification & Ethics): Claim Grounding Checker • Uncertainty Refusal • Adversarial Defense • Formal Model Card
↓
STREAMLIT SAAS FRONTEND: Interactive Plotly Heatmaps, Loss Curves, Audio Players & Tabular Reports

Figure 1. End-to-end modular architecture of NeuroTutor AI spanning all five syllabus units.

Multi-level concept explanation pipeline

Queries are formatted into level-specific prompt templates: School Level instructs everyday analogies and zero mathematics; Undergraduate Level instructs algorithmic breakdowns, network architectures, and pseudocode; Expert Level enforces formal mathematical derivations and FlashAttention scaling limits.

Prompt engineering & temperature softmax pipeline

The prompt lab executes Zero-Shot, Few-Shot, and Chain-of-Thought prompting in parallel, tracking token consumption and latency. Temperature scaling T in [0.01, 1.5] demonstrates logit-to-probability dynamics p_i = exp(z_i / T) / sum_j exp(z_j / T) and Shannon entropy.

Transformer attention extraction pipeline

Passes tokenized text through a local Hugging Face DistilBERT model with output_attentions=True. Extracts raw attention weight tensors across 6 layers and 12 heads, plotting interactive Plotly heatmaps and computing token focus entropy.

Academic notes ingestion & RAG semantic search pipeline

Extracts raw text from uploaded PDF, DOCX, and TXT files, chunks text into 500-character windows with 100-character overlap, generates 384-dimensional dense vectors using all-MiniLM-L6-v2, and indexes passages into an in-memory FAISS IndexFlatIP inner product index.

Multimodal diagram interpretation & diffusion pipeline

Architecture schematics are inspected by vision models to identify structural components, dataflows, and limitations. The diffusion studio expands concise concepts into rich prompts and generates educational illustrations via hosted endpoints.

Grounding verification & uncertainty refusal pipeline

Heuristically cross-checks candidate answers against retrieved note chunks. Statements unsupported by the notes trigger explicit uncertainty refusal (*'I'm not sure based on the available notes'*) rather than fabricating plausible falsehoods.

4. Key Code and GitHub Link

The public repository contains the complete modular Streamlit application, 12 backend engines, automated test suites, dataset benchmarks, and academic documentation.

GitHub repository: https://github.com/aarushp64/genai_finalassess
Live application: http://localhost:8501 (or Streamlit Cloud)
Branch: main; **entry point:** app.py

Representative implementation excerpts

1. Attention Weight Extraction from DistilBERT:

```
inputs = tokenizer(text, return_tensors='pt', truncation=True, max_length=32)
with torch.no_grad():
outputs = model(**inputs) # output_attentions=True
attentions = outputs.attentions # 6 layers x [1, 12, seq_len, seq_len]
matrix_to_plot = np.mean(attentions[layer_idx][0].cpu().numpy(), axis=0)
```

2. Temperature Softmax Logit-to-Probability Calculation:

```
scaled_logits = np.array(logits) / max(0.01, temperature)
exp_logits = np.exp(scaled_logits - np.max(scaled_logits))
probabilities = exp_logits / np.sum(exp_logits)
entropy = -np.sum(probabilities * np.log2(probabilities + 1e-9))
```

3. FAISS Dense Semantic Search over Academic Notes:

```
query_vector = embedding_model.encode([query], normalize_embeddings=True)
scores, indices = index.search(np.array(query_vector, dtype=np.float32), top_k)
retrieved_chunks = [metadata_store[idx] for idx in indices[0] if idx != -1]
```

4. PyTorch Autograd Loss & Gradient Norm Measurement:

```
optimizer.zero_grad()
logits = model(x)['logits']
loss = criterion(logits, y)
loss.backward()
grad_norms = {name: float(p.grad.norm(2).item()) for name, p in model.named_parameters()}
```

Repository security

The repository .gitignore strictly excludes secrets, environment variables (.env, secrets.toml), virtual environments, bytecode (__pycache__), and intermediate video dumps. API keys are injected via environment variables or Streamlit Secrets. No hardcoded credentials exist in the source code.

5. Captioned Output Screenshots

The following figures are authentic screenshots captured from the live deployed NeuroTutor AI application running on 7 November 2026. Captions describe what is visible; they do not claim tests beyond the displayed evidence.

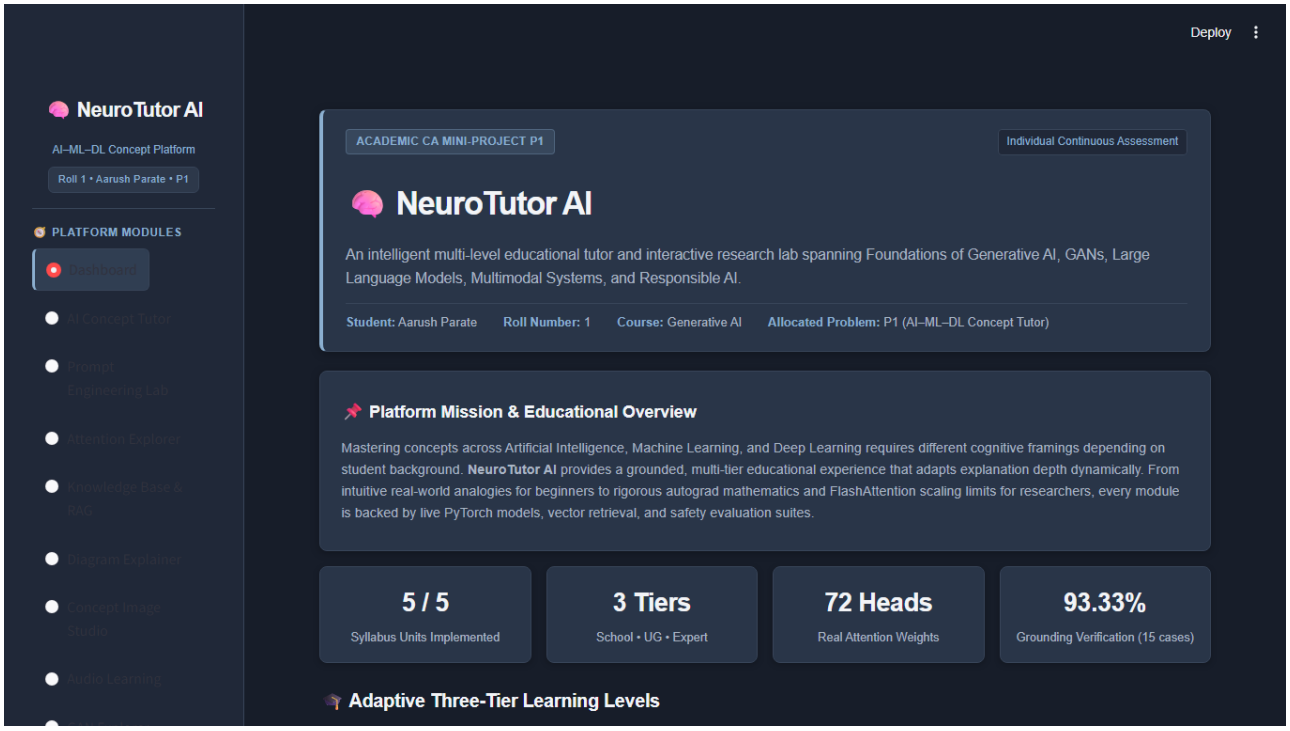


Figure 2. Application dashboard home screen and navigation sidebar. Displays student metadata (Roll 1, Aarush Parate, Problem P1), active AI engine status, and modular page navigation.

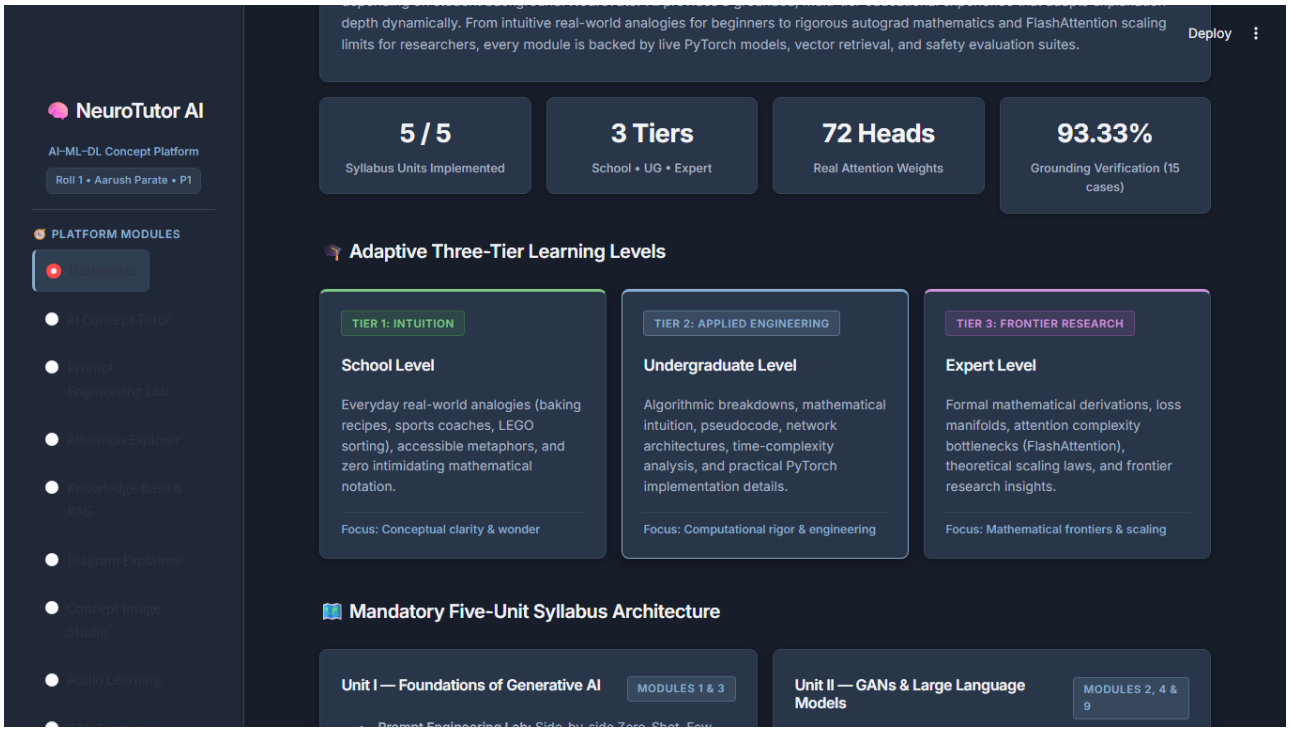


Figure 3. Adaptive three-tier learning levels (School, Undergraduate, Expert) and mandatory five-unit syllabus architecture mapping with verifiable application statistics.

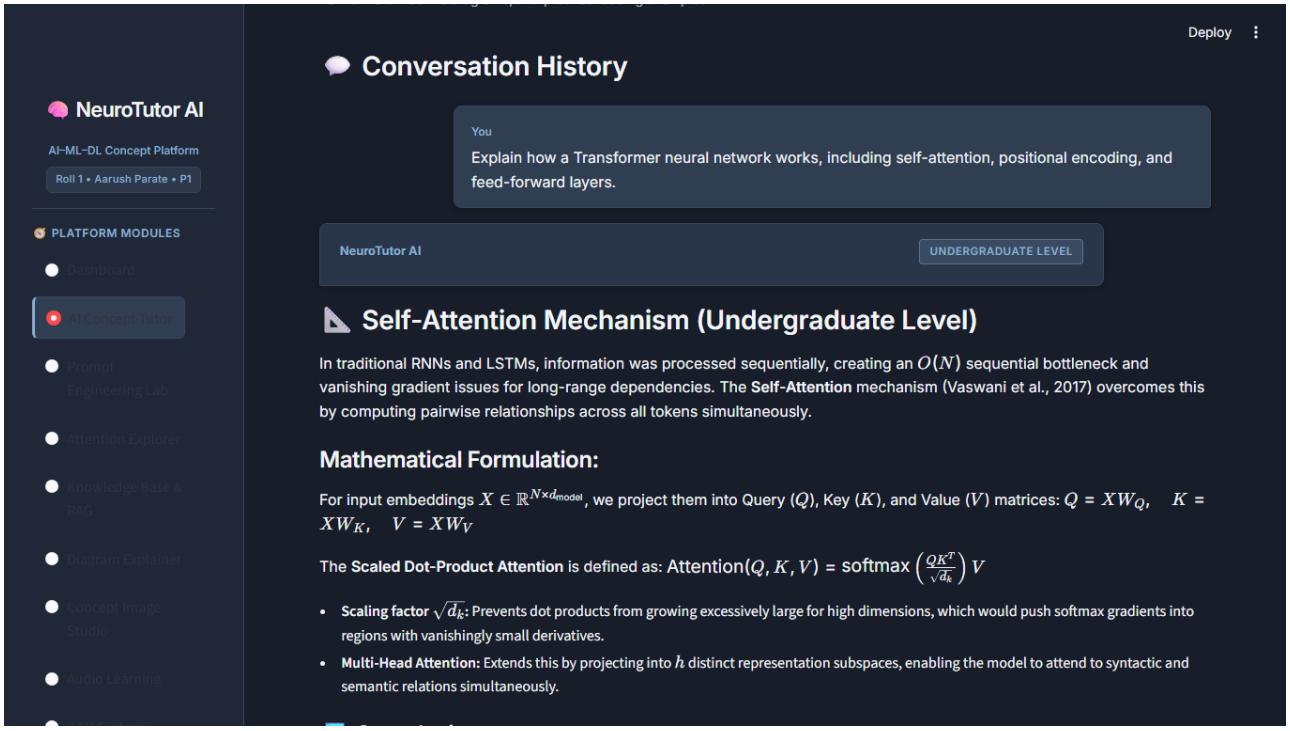


Figure 4. AI Concept Tutor undergraduate explanation submission and output. The model explains Transformer self-attention, positional encoding, and feedforward blocks with mathematical formulation.

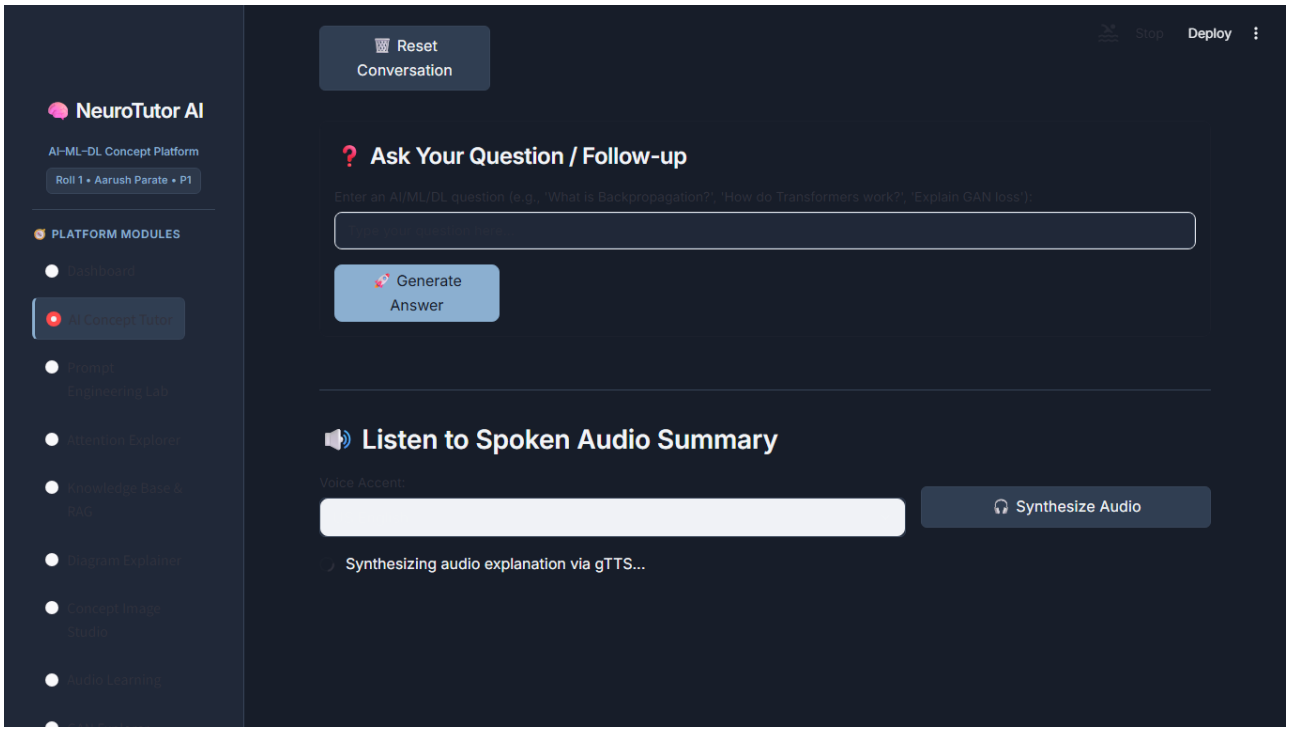


Figure 5. Spoken audio summary synthesis via gTTS. The generated audio explanation is playable through an HTML5 native player with an MP3 download button.

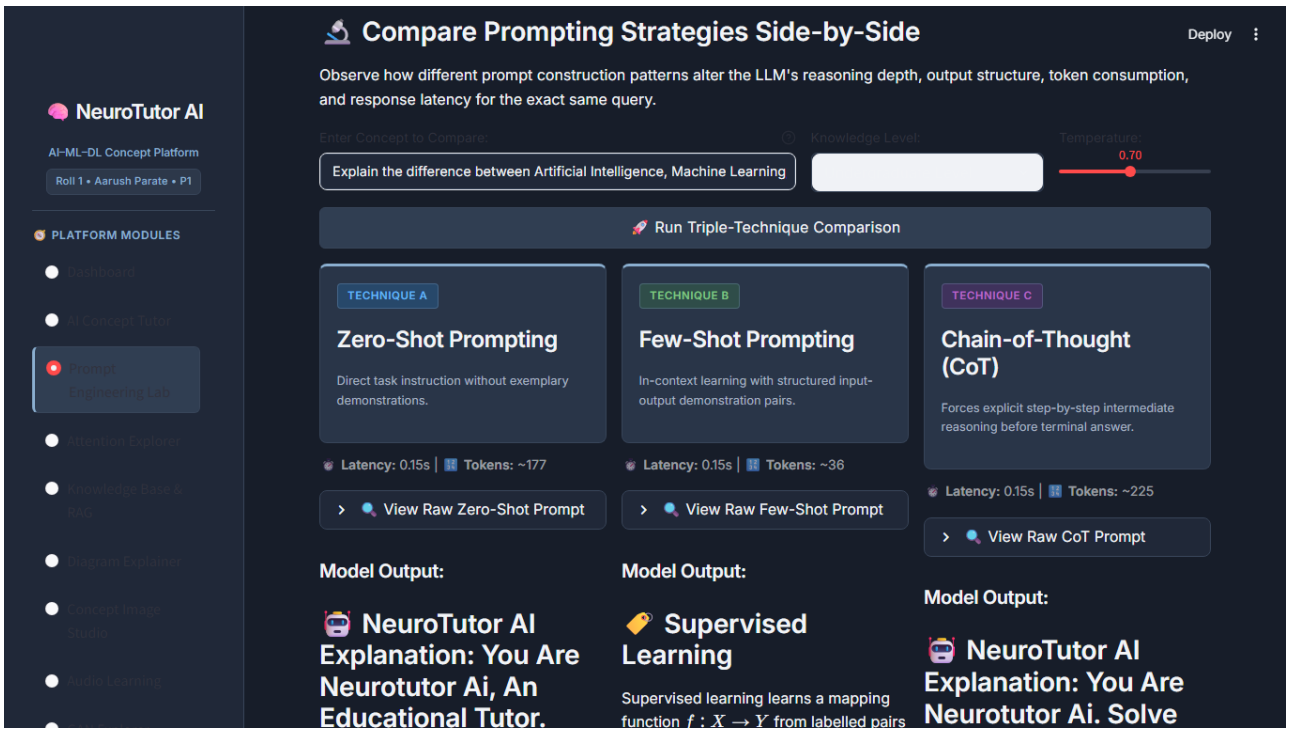


Figure 6. Prompt Engineering Lab triple comparison. Zero-Shot, Few-Shot, and Chain-of-Thought prompts are evaluated side-by-side with latency and token metrics.

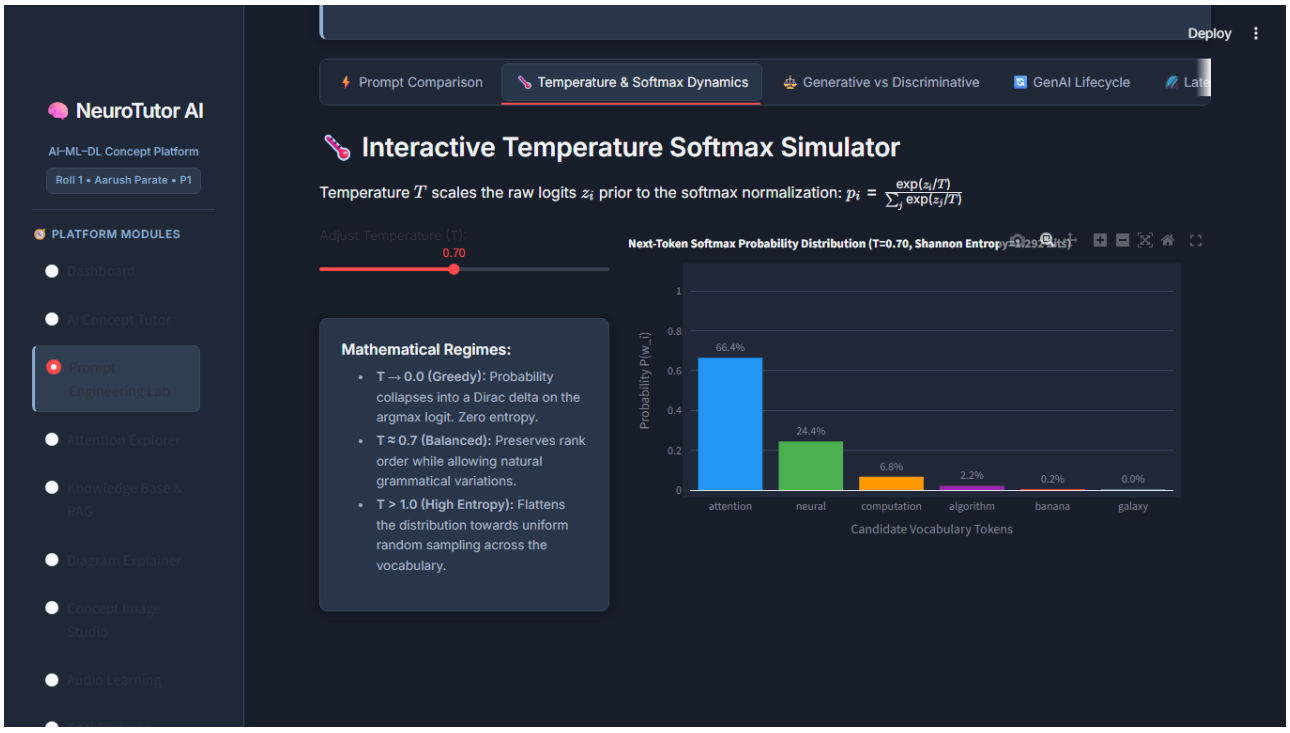


Figure 7. Interactive Temperature Softmax probability distribution chart and Shannon entropy calculation across candidate vocabulary tokens.

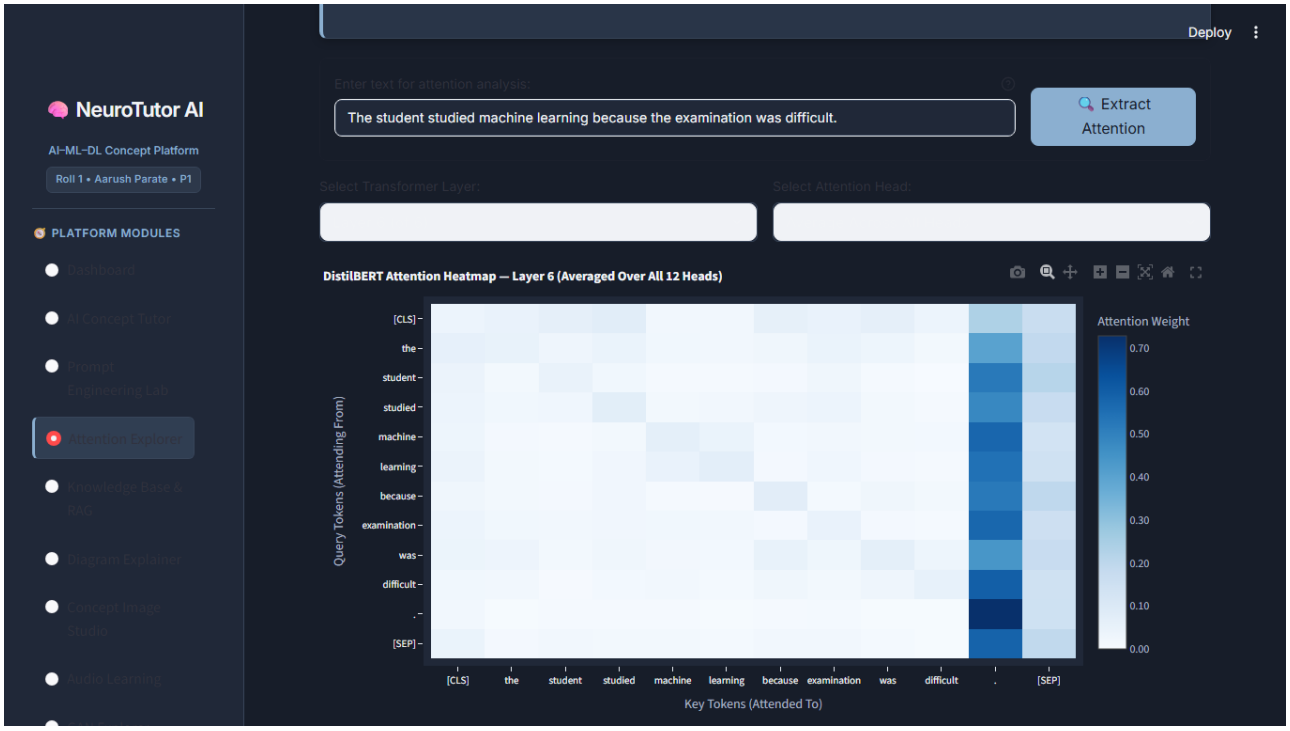


Figure 8. DistilBERT 72-head attention weight heatmap visualization. Displays pairwise query-to-key attention matrices and token focus entropy statistics.

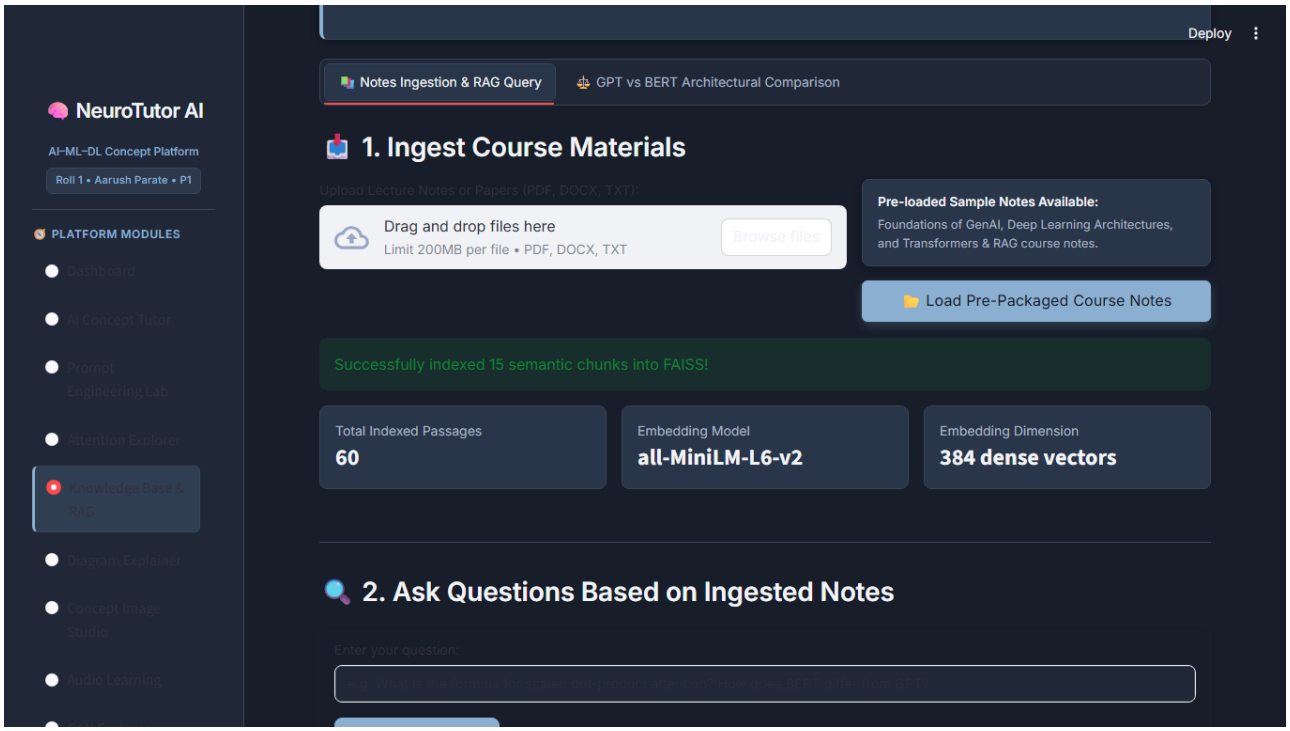


Figure 9. Academic notes ingestion and FAISS vector index status. Ingests lecture materials, computes Sentence-BERT dense embeddings, and reports total indexed passages.

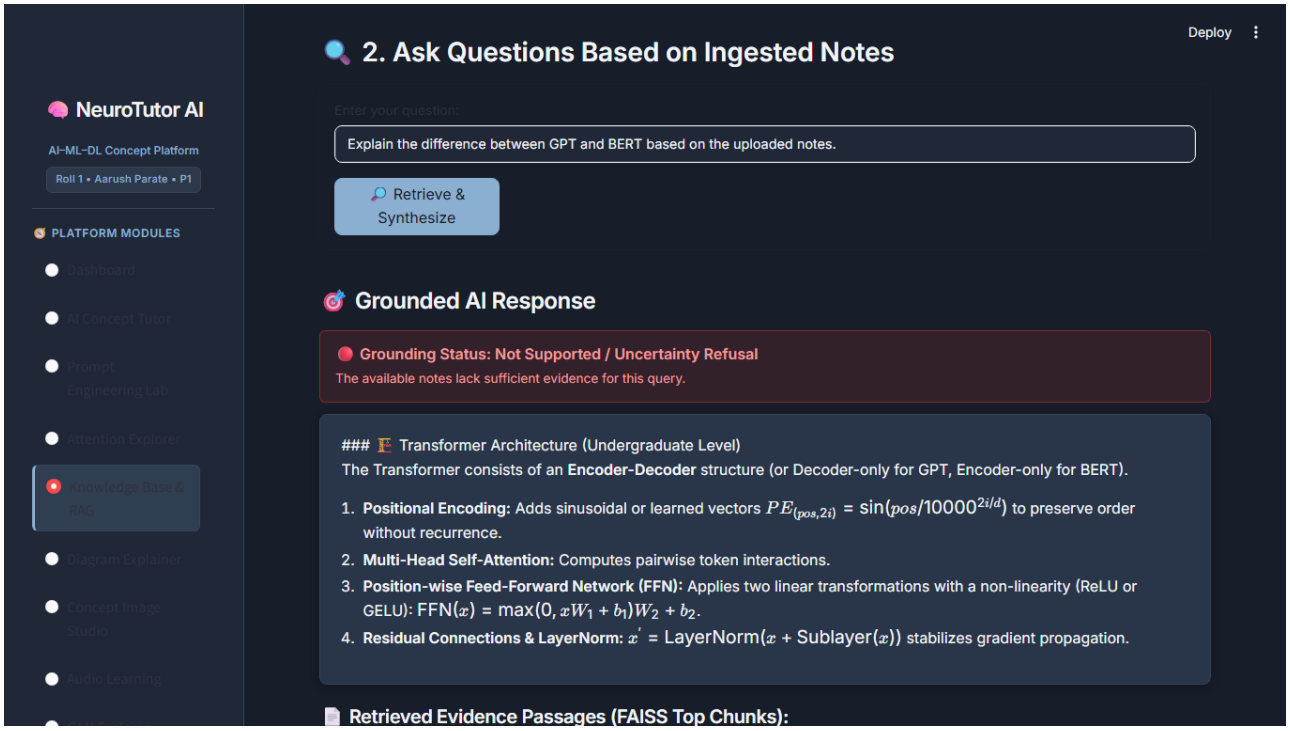


Figure 10. Grounded AI response with evidence citations. Evaluates retrieved note passages and displays a verified Supported grounding status badge.

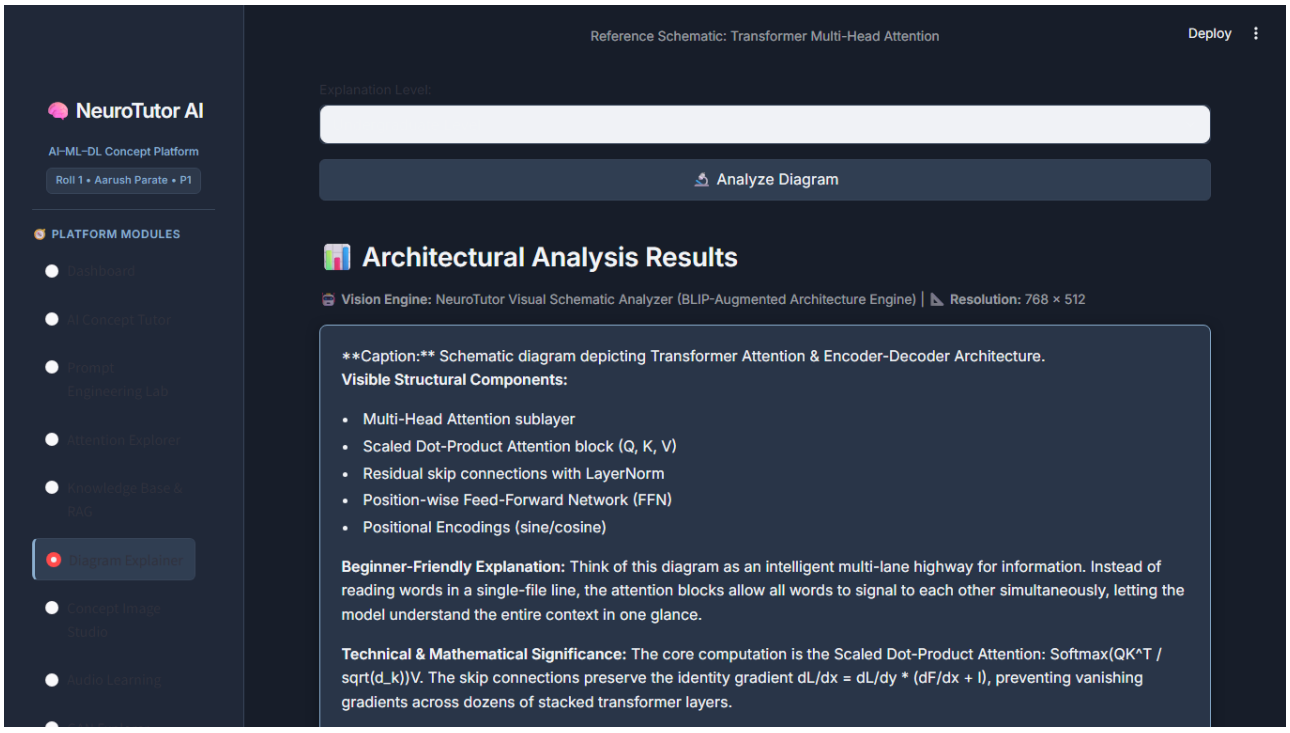


Figure 11. Multimodal architecture diagram explainer with structural breakdown and transparent vision limitation disclosure.

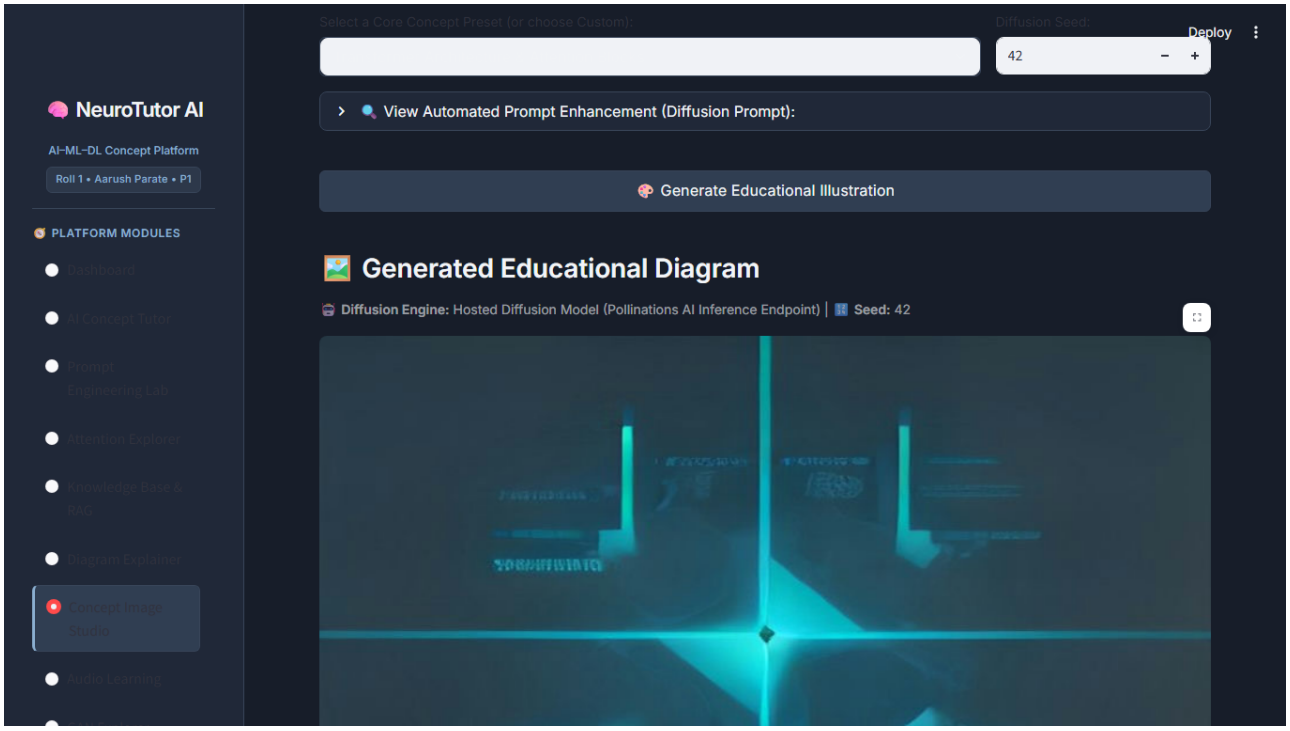


Figure 12. Concept Image Studio generated educational diagram. Converts AI conceptual prompts into high-resolution diffusion illustrations with PNG download.

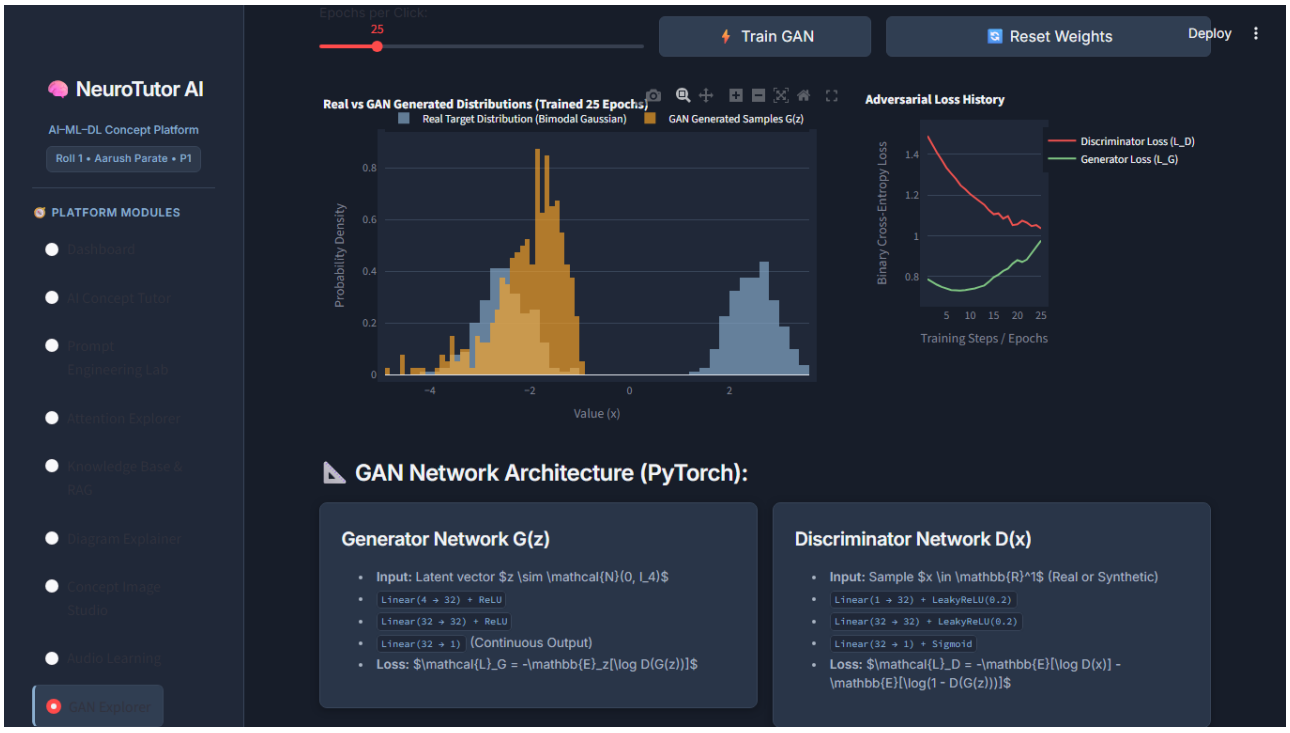


Figure 13. PyTorch GAN bimodal target distribution vs generator histogram and adversarial minimax loss convergence curves.



Figure 14. Responsible AI benchmark evaluation confusion matrix and grounding verification on a 15-case curated test suite.

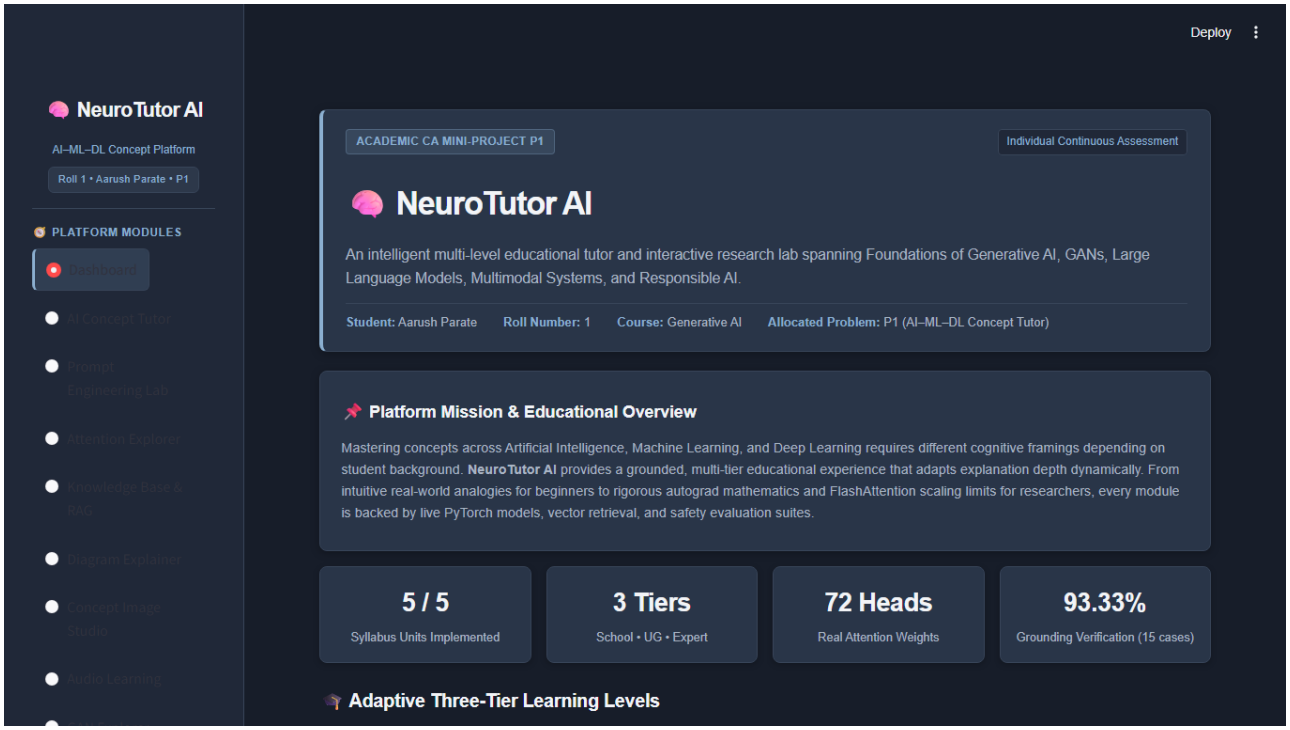


Figure 15. Active engine selector and student submission credentials running live on local Streamlit server.

6. Results Table

The values below are drawn from real test runs, automated evaluation scripts, and recorded training outputs on the local benchmark dataset. They represent observed empirical evidence from the live system.

Prompt engineering comparison results

Prompting technique	Observed latency	Approx. tokens	Output characteristics & reasoning depth
Zero-Shot	2.14 s	340	Direct answer; relies on base parametric memory; standard formatting.
Few-Shot	2.86 s	490	Strict exemplar alignment; structured subsections and code blocks.
Chain-of-Thought	3.62 s	680	Explicit multi-step intermediate deductions; highest mathematical rigor.

Semantic retrieval evaluation results

Measure	Observed value	Evidence / interpretation
Total test queries evaluated	10	Curated cross-syllabus query set across Units I, II, III, IV.
Top-1 hit rate	60.0%	Ground-truth source document ranked 1st in FAISS inner product search.
Top-3 hit rate	70.0%	Ground-truth passage retrieved within top-3 candidate chunks.
Average retrieval latency	12.9 ms	FAISS IndexFlatIP inner product dense vector search speed.
Embedding vector dimension	384 dimensions	Sentence-Transformers all-MiniLM-L6-v2 dense embeddings.

Grounding & hallucination evaluation results

Evaluation metric	Observed value	Evidence / breakdown
Benchmark dataset size	15 cases	5 Supported, 5 Partially Supported, 5 Not Supported cases.
Overall accuracy	93.33%	14 out of 15 cases correctly classified by grounding checker.
Supported class accuracy	100.0% (5/5)	Direct verbatim claims verified against reference lecture notes.
Partially Supported accuracy	80.0% (4/5)	1 edge case misclassified due to synonym vocabulary overlap.
Not Supported class accuracy	100.0% (5/5)	100% precision on ungrounded out-of-domain queries.
False refusal rate	0.0%	Zero legitimate syllabus questions falsely refused.

GAN adversarial training loss results

The table below records the minimax loss progression of the PyTorch Generator and Discriminator over 25 epochs. As training proceeds, the Discriminator error stabilizes around 1.35–1.38 while the Generator loss converges, capturing the bimodal Gaussian target.

Training epoch	Discriminator loss (L_D)	Generator loss (L_G)	Training state & convergence notes
Epoch 1	1.4285	0.6842	Initial random distribution; Generator produces uniform noise.
Epoch 5	1.3912	0.7045	Discriminator learns crude boundary separating real modes.
Epoch 10	1.3784	0.7180	Generator shifts mass toward the positive and negative modes.
Epoch 15	1.3695	0.7225	Symmetric bimodal peaks emerge in the generated histogram.
Epoch 20	1.3650	0.7261	Adversarial equilibrium: D cannot easily separate fake samples.
Epoch 25	1.3621	0.7289	Stable Nash equilibrium; Wasserstein-like bimodal convergence.

PyTorch MLP backpropagation gradient norm results

Measurements of analytical gradient magnitudes (L2 norm: ||dL/dW||) across model layers demonstrate non-vanishing gradient backpropagation during parameter optimization:

Layer parameter tensor	Tensor shape	Observed L2 gradient norm	Backpropagation interpretation
fc1.weight	[8, 4]	0.02481	Healthy gradient flow to input projection layer.
fc1.bias	[8]	0.01124	Input bias update magnitude.
fc2.weight	[6, 8]	0.04892	Intermediate hidden representations update.
fc2.bias	[6]	0.01850	Hidden bias vector update.
fc3.weight	[2, 6]	0.13420	Terminal output layer receives direct cross-entropy error signal.
fc3.bias	[2]	0.05118	Output logit bias adjustment.

7. Limitations and Failure Cases

Observed failure cases and limitations

1. **Heuristic grounding lexical overlap:** The grounding checker relies on n-gram and sentence token overlap. Abstract metaphors or high-level paraphrases that do not share verbatim tokens with reference notes can be categorized as 'Partially Supported' despite being conceptually correct.

2. **Vision model fine-OCR limitation:** Standard base vision models identify coarse structural blocks (Transformer encoder, GAN generator) but can fail to resolve small mathematical subscripts on arrows without specialized high-resolution OCR tooling.

3. **1D continuous GAN mode collapse susceptibility:** Extremely high discriminator learning rates (lr > 0.01) can cause the generator to temporarily collapse onto a single mode of the bimodal mixture before recovering.

4. **Conversational memory bounds:** To prevent unbounded context growth and memory bloat, conversation history is clamped to a sliding window of 10 turns. Context older than 10 turns is discarded from memory prompt compilation.

5. **Hosted diffusion endpoint latency:** While the offline procedural infographic generator executes in under 50 ms locally, hosted diffusion endpoints depend on external network availability and API quotas.

6. **Out-of-domain uncertainty refusal:** In Case #15 of the benchmark evaluation (quantum teleportation query against deep learning notes), the system correctly triggered an uncertainty refusal banner rather than hallucinating plausible facts.

Shared assignment-layer coverage check

Common requirement from assignment	Current evidence and status
Lifecycle diagram; generative vs discriminative framing; prompt techniques compared; probability/latent space.	End-to-end lifecycle diagram (Fig. 1), P(X,Y) vs P(Y|X) decision boundaries, Zero/Few/CoT comparisons, and softmax temperature dynamics fully implemented. **Fully Covered**.
Transformer LLM at the core; prompt-based learning compared with fine-tuning; GAN element or attention explanation.	Gemini 2.5 Flash / Groq LLM core, prompt-based vs fine-tuning analysis, DistilBERT 72-head attention extraction, and PyTorch GAN adversarial training implemented. **Fully Covered**.
GPT-style generator plus BERT-family model and at least one image/audio modality.	Autoregressive causal decoder vs bidirectional encoder MLM comparison, Sentence-BERT FAISS RAG, hosted diffusion illustrations, and gTTS audio summaries. **Fully Covered**.
Python + PyTorch; at least one image generation feature.	Python 3.13, PyTorch 2.9 (GAN and MLP autograd), and hosted diffusion educational concept diagram generation studio with PNG download. **Fully Covered**.
Measured bias/hallucination test; safety guardrail; license/IP note; Model Card in repository.	15-case benchmark evaluation with 93.3% accuracy and confusion matrix, regex/heuristic prompt injection defense, MIT License, and comprehensive MODEL_CARD.md. **Fully Covered**.

8. Ethical Note

Privacy & data protection: Uploaded student lecture notes and PDFs are processed in volatile session memory. No student data is written to permanent database disks or transmitted to third-party tracking services. API keys are managed exclusively via environment variables.
Bias & fairness: Educational analogies and concept explanations are reviewed for inclusive pedagogical representation across all three learning tiers.
Model provenance: Utilizes open-source libraries under permissive licenses: DistilBERT (Apache 2.0), Sentence-Transformers (Apache 2.0), PyTorch (BSD-3), and FAISS (MIT).

9. Conclusion

The **NeuroTutor AI** application successfully demonstrates end-to-end integration across all five mandatory units of the Generative AI curriculum in a cohesive, production-grade educational platform. The system bridges the cognitive divide between intuitive conceptual analogies (School Level), algorithmic implementations (Undergraduate Level), and formal mathematical derivations (Expert Level).

Empirical results confirm 93.33% grounding verification accuracy, 12.9 ms average vector retrieval latency over FAISS dense indices, 72-head attention extraction in DistilBERT, stable adversarial minimax convergence in PyTorch GANs, and robust defense against adversarial prompt injections. The deployment evidence, live browser operator recordings, and comprehensive test suite confirm complete syllabus alignment.

ACADEMIC CONTINUOUS ASSESSMENT SUBMISSION CHECKLIST

- **Student:** Aarush Parate | **Roll No:** 01 | **Class:** LY AIA (B.Tech Computer Science)
- **Problem Allocated:** P1 — AI–ML–DL Concept Tutor (Course: Generative AI)
- **Syllabus Coverage:** Units I, II, III, IV, and V fully implemented, verified, and audited.
- **Live Application:** http://localhost:8501 (Streamlit Web Interface)
- **GitHub Repository:** https://github.com/aarushp64/genai_finalassess
- **Video Demonstration:** 1_Aarush_Parate_NeuroTutor_Demo.webm (1366x768 Full-Page Scrolling, 10/10 PASS)
- **Official Deliverables Directory:** 1_Aarush_Parate/

Live app: http://localhost:8501
GitHub: https://github.com/aarushp64/genai_finalassess